MMDU MULLANA OFFICIAL SYLLABUS • BCSE-004 • UNIT 1

Unit 1: Introduction to Programming and Python (Architecture, Tokens & Operators)

Subject: Python Programming (BCSE-004) • **Exam Weightage:** 24 - 28 Marks • **Study Time:** 35 mins

Algorithms, machine vs high-level languages, compiled vs interpreted execution, Python philosophy (PEP 20), CPython compiler & PVM architecture, variable reference model vs C box model, indentation rules, keywords, literals, formatted I/O, and 9-tier operator precedence.

1. Language Hierarchy & CPython Execution Architecture

1. Program, Algorithm & Language Classification

A **program** is a sequence of deterministic instructions executed by a computer to solve a problem. An **algorithm** is the logical foundation of a program: a finite, unambiguous, step-by-step procedure accepting specified inputs and yielding a well-defined output in finite time.

Language Generation	Level	Translation Mechanism	Portability	Examples
Machine Code (1GL)	Binary (0s & 1s)	Executed directly by CPU microcode	None	`10110000 01100001`
Assembly (2GL)	Low (Mnemonics)	Translated via *Assembler*	Low	`MOV AX, 1; ADD BX, AX`
Compiled (3GL)	High-level	Ahead-Of-Time (AOT) to machine code	Source-portable only	C, C++, Rust
Interpreted (4GL)	Very High-level	Bytecode interpretation in Virtual Machine	**100% Portable**	**Python**, JavaScript

2. CPython Architecture: Compiler & Python Virtual Machine (PVM)

Python uses a **two-tier hybrid execution pipeline**:

1. **Compilation Step:** The CPython compiler parses source code (`.py`) into an Abstract Syntax Tree (AST) and translates it into platform-independent intermediate **Bytecode** (`.pyc` files cached inside the `__pycache__` directory).

2. **Execution Step:** The **Python Virtual Machine (PVM)** is the runtime engine of CPython. It loops through the bytecode instructions line-by-line and maps them to native OS/CPU system calls.

> ⚡ **KEY TAKEAWAY:**
> **KEY TAKEAWAY & BOTTOM LINE:**

> Python is neither purely compiled nor purely interpreted. It compiles source code to Bytecode once and interprets Bytecode inside the PVM, combining the portability of interpretation with the speed of bytecode.

2. Variable Reference Model, Tokens & Operator Precedence

1. Variables as Named References (Pointers)

In C, a variable is a named memory box storing raw values directly. In Python, variables are **symbolic references (pointers)** bound to dynamic heap objects:

```
a = 256
b = 256
print(a is b)        # True (Integer interning cache for range -5 to 256)
print(id(a), id(b))  # Identical memory addresses

x = [1, 2, 3]
y = x                # y points to the SAME list object in heap
y.append(4)
print(x)             # Output: [1, 2, 3, 4] (x reflects mutation!)
```

2. Complete 9-Tier Operator Precedence Matrix

Rank	Operator Group	Symbols	Associativity
1	Parentheses & Indexing	`()`, `[]`, `{}`, calls, slicing	Left-to-Right
2	Exponentiation		Right-to-Left
3	Unary Operators	`+x`, `-x`, `~x` (Bitwise NOT: `~x = -x-1`)	Right-to-Left
4**	Multiplicative	`*`, `/` (float div), `//` (floor div), `%` (mod)	Left-to-Right
5	Additive	`+`, `-`	Left-to-Right
6	Bitwise Shifts	`<<`, `>>`	Left-to-Right
7	Bitwise Logic	`&` (AND) → `^` (XOR) →	(OR)	Left-to-Right
8	Comparisons & Identity	`==`, `!=`, `<`, `>`, `<=`, `>=`, `is`, `in`	Left-to-Right
9	Boolean Logic	`not` → `and` → `or` (Short-circuit)	Left-to-Right

💡 **TOPPER MEMORY HACK:**
TOPPER EXAM TIP & QUESTION BLUEPRINT:

> **Why Python has no `++` or `--`:** Integers in Python are immutable heap objects. In-place increment would violate object immutability. Writing `++x` is parsed as two unary positives `+(+(x))`, evaluating to `x`. Always use `x += 1`.